



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2413 - BASES DE DATOS

Examen

Fecha: 8 de julio de 2026

1^{er} semestre 2026 - Profesores: Eduardo Bustos - Christian Álvarez

INSTRUCCIONES

- a) La prueba tiene 4 preguntas, cada una vale 15 puntos ($60/10 + 1 = 7$) y una pregunta Bonus de 5 puntos sobre la nota final de la prueba. **Duración 2,5 horas.**
- b) **Lea toda la prueba primero**, las preguntas no están en orden de dificultad
- c) Al finalizar la prueba debe digitalizar cada pregunta en formato PDF VERTICAL y subirlas a Canvas **correctamente.**
- d) Cada plana debe contener su Nombre Completo y Número de Lista en la parte superior.
- e) Responda **cada pregunta en una hoja distinta**, puede usar anverso y reverso, no mezcle respuestas de diferentes preguntas.
- f) Debe firmar la **lista de asistencia.**
- g) No seguir estas instrucciones dificulta el trabajo de evaluación y por ende el tiempo de entrega de las notas, además los expone a un descuento de hasta 5 décimas de la nota final.

Pregunta 1 - Modelo ER + Diseño de BD + FN

Una organización de salud requiere automatizar la gestión de su banco de sangre. Un banco de sangre provee bolsas de sangre a los pacientes que la necesitan para sus procedimientos. La sangre se recibe exclusivamente de donaciones realizada por personas donantes. Las reglas de negocio son:

- Cada **donante** se registra con su RUN, nombres, apellidos, fecha de nacimiento, sexo, tipo de sangre (A+, A-, B+, B-, AB+, AB-, O+, O-), teléfono y dirección. Un donante puede realizar múltiples **donaciones** a lo largo del tiempo.
- Cada **donación** tiene un código único, fecha, hora, volumen extraído (ml), y debe registrar al **personal médico** (enfermero/a o médico) que la atendió. Toda donación debe pasar por un **examen de seguridad** (o tamizaje serológico) antes de ser aceptada (resultado: apta/no apta, y motivo en caso de rechazo).
- La sangre donada se almacena en **bolsas de sangre**, cada una con código, fecha de vencimiento, tipo de sangre, componente (sangre completa, plasma, glóbulos rojos, plaquetas) y estado (disponible, reservada, usada, vencida). Una donación puede generar una o más bolsas (por fraccionamiento en componentes).
- Los **pacientes receptores** se registran con RUN, nombres, apellidos, fecha de nacimiento, tipo de sangre, diagnóstico y teléfono de contacto. Un paciente puede generar múltiples **solicitudes de sangre**, indicando tipo de sangre requerido, componente, cantidad de bolsas solicitadas y urgencia (alta, media, baja). Cada solicitud debe registrar además al **médico tratante** que la autoriza.
- Cada solicitud puede ser satisfecha con una o varias bolsas de sangre, y una bolsa solo puede destinarse a la solicitud de un paciente (es decir, a un único paciente receptor).
- Se debe registrar la **transfusión** propiamente dicha: fecha, hora, bolsa(s) utilizada(s), paciente receptor y personal médico que la realiza. Después de la transfusión, el estado de la bolsa cambia a “usada”. Cada solicitud genera una transfusión.
- El personal médico se registra con RUN, nombre, especialidad y turno de trabajo.
- Se debe llevar control de **eventos adversos**, tanto en donantes (post-donación) como en pacientes receptores (post-transfusión), registrando fecha, descripción y gravedad.

Se pide desarrollar lo siguiente:

a) Modelo Entidad-Relación (10 puntos)

Construya el diagrama E-R correspondiente, identificando entidades, atributos (incluyendo claves primarias), relaciones y cardinalidades. Señale claramente si existen entidades débiles. Evalúe si es necesario usar herencia. Use notación Chen o Ramakrishnan.

b) Esquema de la Base de Datos (5 puntos)

A partir del modelo E-R, obtenga el esquema relacional (paso de modelo E-R a modelo relacional), indicando tipos de datos, claves primarias (PK), claves foráneas (FK) y restricciones de integridad relevantes. Justifique si su modelo está en 3NF o BCNF.

Pregunta 2 - SQL + Lógica en la BD

Una empresa de movilidad urbana arrienda scooters eléctricos. Los scooters se distribuyen en distintas estaciones de la ciudad, y los usuarios los arriendan a través de una app, iniciando el viaje en una estación y devolviéndolo en otra (o en la misma). El costo del viaje tiene una componente fija al desbloquear el scooter y un costo por minuto de uso.

```
CREATE TABLE ESTACION (  
    cod_estacion      INT PRIMARY KEY,  
    nombre            VARCHAR(50) NOT NULL,  
    comuna            VARCHAR(50) NOT NULL,  
    capacidad_max     INT NOT NULL CHECK (capacidad_max > 0)  
);  
  
CREATE TABLE SCOOTER (  
    cod_scooter       INT PRIMARY KEY,  
    modelo            VARCHAR(30) NOT NULL,  
    nivel_bateria     INT NOT NULL CHECK (nivel_bateria BETWEEN 0 AND 100),  
    estado            VARCHAR(20) NOT NULL CHECK (estado IN  
(  
'disponible', 'arrendado', 'mantencion', 'fuera_servicio'  
)),  
    cod_estacion      INT NOT NULL REFERENCES ESTACION(cod_estacion)  
);  
  
CREATE TABLE USUARIO (  
    run_usuario       VARCHAR(12) PRIMARY KEY,  
    nombres           VARCHAR(50) NOT NULL,  
    apellidos         VARCHAR(50) NOT NULL,  
    correo            VARCHAR(80) UNIQUE NOT NULL,  
    saldo             DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (saldo >= 0)  
);  
  
CREATE TABLE TARIFA (  
    cod_tarifa        INT PRIMARY KEY,  
    nombre            VARCHAR(30) NOT NULL,  
    valor_desbloqueo  DECIMAL(10,2) NOT NULL,  
    valor_por_minuto  DECIMAL(10,2) NOT NULL  
);  
  
CREATE TABLE ARRIENDO (  
    cod_arriendo      INT PRIMARY KEY,  
    run_usuario       VARCHAR(12) NOT NULL REFERENCES USUARIO(run_usuario),  
    cod_scooter        INT NOT NULL REFERENCES SCOOTER(cod_scooter),  
    cod_tarifa         INT NOT NULL REFERENCES TARIFA(cod_tarifa),  
    cod_estacion_inicio INT NOT NULL REFERENCES ESTACION(cod_estacion),  
    cod_estacion_fin   INT REFERENCES ESTACION(cod_estacion),  
    fecha_inicio       DATETIME NOT NULL,  
    fecha_fin          DATETIME,  
    costo_total        DECIMAL(10,2)  
);  
  
CREATE TABLE MANTENIMIENTO (  
    cod_mantenimiento INT PRIMARY KEY,  
    cod_scooter        INT NOT NULL REFERENCES SCOOTER(cod_scooter),  
    fecha              DATETIME NOT NULL,
```

```
descripcion          VARCHAR(200),
costo                 DECIMAL(10,2)
);
```

Notas sobre el modelo:

- Un arriendo con `fecha_fin` y `cod_estacion_fin` en NULL representa un viaje **en curso**.
- El campo `costo_total` se calcula al finalizar el arriendo (no se ingresa manualmente).
- Un scooter con `nivel_bateria` bajo un umbral, o marcado en `mantencion` o `fuera_servicio`, no debería poder arrendarse.

Datos de ejemplo (para probar sus consultas)

```
INSERT INTO ESTACION VALUES (1, 'Plaza Italia', 'Providencia', 20), (2, 'Los Leones',
                              'Providencia', 15), (3, 'Baquedano', 'Santiago', 25);

INSERT INTO TARIFA VALUES (1, 'Normal', 500, 80), (2, 'Estudiante', 300, 60);

INSERT INTO SCOOTER VALUES
(101, 'Xiaomi Pro2', 85, 'disponible', 1),
(102, 'Xiaomi Pro2', 15, 'disponible', 1),
(103, 'Ninebot Max', 60, 'arrendado', 2),
(104, 'Ninebot Max', 0, 'mantencion', 3);

INSERT INTO USUARIO VALUES
('11111111-1', 'Ana', 'Soto', 'ana@mail.com', 5000),
('22222222-2', 'Luis', 'Perez', 'luis@mail.com', 1200);

INSERT INTO ARRIENDO VALUES
(1, '11111111-1', 103, 1, 2, NULL, '2026-07-01 09:00:00', NULL, NULL);
```

a) Construya las consultas SQL para (6 puntos — 3 consultas, 2 puntos c/u)

- I) Listar el nombre y correo de los usuarios que tienen **saldo insuficiente** (menor a \$2.000) para iniciar un arriendo con tarifa “Normal”.
- II) Obtener, por cada estación, la **cantidad de scooters disponibles** actualmente, ordenado de mayor a menor.
- III) Calcular el **ingreso total** generado por cada tarifa, considerando solo arriendos ya finalizados (`fecha_fin IS NOT NULL`).

b) Cree el Procedimiento Almacenado (4 puntos)

Llamado `finalizar_arriendo(cod_arriendo, cod_estacion_fin)`: cierra un arriendo en curso, calculando `costo_total` en función del tiempo transcurrido y la tarifa asociada (`valor_desbloqueo + valor_por_minuto * minutos`), actualiza `fecha_fin`, y descuenta el costo del saldo del usuario. Considere validar que el arriendo exista y esté efectivamente en curso (`fecha_fin IS NULL`).

c) Cree un Trigger (3 puntos)

Que, al **insertar** un nuevo registro en la tabla `ARRIENDO`, cambie automáticamente el estado del scooter correspondiente a `'arrendado'`.

d) Cree una Vista (2 puntos)

Llamada `VW_SCOOTERS_ESTADO` que muestre, para cada scooter: código, modelo, estado, nombre de la estación actual, comuna, y un campo calculado `requiere_atencion` ('SI'/'NO') que indique si el scooter tiene batería bajo 20 % o está en estado `mantencion/fuera_servicio`.

Pregunta 3: Transacciones, Conflict Serializability y Locks

Considere el siguiente schedule con tres transacciones T_1 , T_2 y T_3 que operan sobre los elementos A , B y C de la base de datos:

T1	T2	T3
$R(A)$		
	$R(B)$	
	$W(A)$	
		$R(B)$
$W(B)$		
		$W(A)$
	$R(C)$	
	$W(C)$	

a) Identificación de conflictos (4 puntos)

Identifique **todos los pares de operaciones conflictivas** en el schedule anterior. Para cada par encontrado, indique:

- Las transacciones involucradas y el elemento sobre el que actúan.
- El tipo de conflicto (Lectura-Escritura, Escritura-Lectura o Escritura-Escritura).
- Qué anomalía podría producirse en la práctica (lectura sucia, lectura no repetible, o actualización perdida), justificando brevemente.

b) Conflict Serializable (8 puntos)

- Construya el **grafo de precedencia** para el schedule dado, indicando claramente cada arista y la(s) operación(es) conflictiva(s) que la justifican. *(4 puntos)*
- En base al grafo, determine si el schedule es **conflict serializable**. Si lo es, indique un orden serial equivalente $T_i; T_j; T_k$. Si no lo es, señale el ciclo que lo impide. *(2 puntos)*
- Verifique su respuesta anterior usando el **método de permutaciones**, mostrando y explicando al menos 2 intercambios de operaciones conflictivas o no conflictivas. *(2 puntos)*

c) Two-Phase Locking - 2PL (3 puntos)

Suponga ahora que T_1 y T_2 ejecutan bajo el protocolo **Strict 2PL**, con el siguiente orden de solicitudes de locks:

T1	T2
Solicita shared lock sobre A	
	Solicita exclusive lock sobre B
Solicita exclusive lock sobre B	
	Solicita exclusive lock sobre A

- Indique paso a paso qué ocurre con cada solicitud (otorgado, en espera, etc.). *(2 puntos)*
- ¿Se produce un **deadlock**? Justifique señalando el ciclo de espera, o explique por qué no ocurre. *(1 punto)*

Pregunta 4: EEDD, Almacenamiento e Índices

Sea $R(w, x, y, z)$ una tabla con **2.400.000 tuplas**. Cada página almacena B tuplas; el índice B+ almacena P punteros por página; la memoria tiene capacidad para M páginas.

a) Índice Hash Unclustered (5 puntos)

Existe un **índice Hash Unclustered** sobre el atributo w . ¿Cuál es el costo de I/O de la siguiente consulta? Justifique, considerando que existen en promedio **4 tuplas** con el mismo valor de w , distribuidas en páginas de datos distintas.

```
SELECT x, y FROM R WHERE w = 358
```

b) Índice B+ Clustered (5 puntos)

Existe un **índice B+ Clustered** sobre el atributo x . ¿Cuál es el costo de I/O de la siguiente consulta? Justifique, considerando que aproximadamente **1/8** de las tuplas satisfacen la condición.

```
SELECT w, z FROM R WHERE x < 200
```

c) Responda V o F y justifique (5 puntos)

- i) Un índice Hash Unclustered requiere, en el peor caso, un acceso a disco por cada tupla que cumple la condición de igualdad, además de los accesos al propio índice.
- ii) Un índice B+ Clustered es eficiente tanto para búsquedas de igualdad como para consultas de rango.
- iii) En un índice Unclustered, el orden físico de las tuplas en disco coincide con el orden del índice.
- iv) La altura de un índice B+ depende del número de punteros por página P y del número de hojas necesarias para indexar todas las tuplas.
- v) Una tabla puede tener a lo más un índice Clustered, pero múltiples índices Unclustered.

Bonus: Recuperación de Fallas

Responda Verdadero o Falso a las siguientes afirmaciones (no debe justificar).

- a) RAID es un mecanismo para soportar fallas de equipos en Sistemas Distribuidos
- b) En Undo Logging el log registra el valor nuevo de cada escritura, para recuperarse en caso de fallas catastróficas.
- c) Nonquiescent Checkpoints son un tipo de checkpoint que no requiere *detener* el sistema.
- d) Redo Logging funciona *al revés* que Undo Logging.
- e) Las copias distribuidas sirven para recuperarse de fallas catastróficas

Torpedo

SQL

Sintaxis consulta SQL

```
SELECT [ALL | DISTINCT] * | [COUNT, MAX, MIN, SUM, AVG](col1), col2, ...
      [AS alias_column]
FROM table1
      [JOIN table2 ON table1.common_field = table2.common_field]
WHERE condition1 [AND | OR condition2]
      [BETWEEN value1 AND value2]
      [IN (value1, value2, ...)]
      [LIKE pattern]
      [= | <> | <= | >= | < | >] value
GROUP BY column1, column2, ...
HAVING aggregate_condition [COUNT, MAX, MIN, SUM, AVG]
ORDER BY column1 [ASC | DESC]
UNION | INTERSECT | EXCEPT
[SELECT column1 FROM another_table]
LIMIT row_count;
```

Sintaxis Stored Procedure

```
CREATE OR REPLACE FUNCTION <nombre_función> (<argumentos>)
RETURNS [VOID | TABLE(...) ] AS $$
DECLARE
    <declaración_variables>
BEGIN
    <sentencias_SQL>
END; $$ language plpgsql
```

Sintaxis PL/pgSQL

```
BEGIN
    IF <condición> THEN
        <cuerpo_if>
    ELSE
        <cuerpo_else>
    END IF;
    INSERT INTO <tabla> VALUES (<tupla_valores>);
    FOR <variable> IN <iterable>
    LOOP
        <cuerpo_loop>
    END LOOP;
    RETURN QUERY <consulta_SQL>;
END;
```